DEPARTMENT OF COMPUTER SCIENCE

Senior Capstone Project Proposal

LitterBot: A Self-Driving Sidewalk Robot That Finds and Maps Campus Litter

Student Name(s):	Ethan Bucy
Team / Group Number (if applicable):	N/A (individual project)
Faculty Advisor:	TBD
Course / Section:	CS 480W
Submission Date:	September 21, 2026

1. Problem Statement and Motivation

Describe, in plain language, the real problem or unmet need your project addresses and why it matters to actual users or stakeholders.

> ***Consider:*** *who experiences this problem today, how they currently cope without your solution, and what evidence (personal experience, articles, data, interviews) suggests the problem is real and worth solving.*

Litter on campus sidewalks gets cleaned up only when someone happens to notice it. Grounds staff find it by walking routes or answering complaints, students who would pick it up don't know where it is, and nobody records where litter builds up over time. Small items (bottles, cans, cups, wrappers) can sit for days.

Who is affected: grounds/Facilities staff (time spent searching instead of cleaning), students and visitors (a less pleasant campus), and anyone placing bins or planning cleanups (no location data).

Evidence: my own daily observation, and the popularity of apps such as Litterati [6], whose users photograph and geotag litter by hand. That shows demand for litter location data, and also that it depends entirely on people noticing litter first. **Open question:** in my testing-permission request I will ask Facilities what would make an automatic report useful to them.

Personal motivation: the project covers the core pieces of an autonomous vehicle (perception, control, safety) at student scale, which I plan to study in a master's program.

2. Project Overview

Summarize, in 3–5 sentences, what you plan to build or investigate, as if explaining it to a classmate outside the CS department.

> ***Consider:*** *naming the type of artifact (web app, mobile app, ML model, systems tool, research study, etc.) and the single core function it must perform.*

I will build a shoebox-sized, four-wheeled robot that drives one campus sidewalk route by itself at walking pace. A camera and a small neural network (a model trained on example photos) spot litter, and the robot saves a photo with its GPS location. When it next has internet, it uploads those finds to a public website with a map. After I approve each photo, anyone sent there by QR-code posters can see the spot and mark it "cleaned."

Artifact type: robot + two machine-learning models + web app. **Core function:** turn litter on a sidewalk into a pin on a public map.

3. Goals and Objectives

List the specific, measurable objectives your finished project should achieve, distinguishing must-have goals from stretch goals.

> ***Consider:*** *phrasing each objective so a reader could later check yes/no whether it was met (e.g., "users can upload a file under 10MB and receive results in under 5 seconds").*

Every goal is checkable yes/no. The must-haves are the minimum bar: if they are met, the project is complete even if self-driving falls short.

Must-have (minimum bar, by Feb 13, 2027), robot driven by gamepad

- Safety: releasing the hold-to-run button stops the robot within 0.5 m in 10 of 10 trials.
- Detection accuracy: the litter model scores mAP50 ≥ 0.50 (target 0.65) on held-out campus photos. mAP50 is a 0–1 score for finding objects and boxing them roughly in the right place.

- Field detection: of 20 staged litter items on the route, ≥ 12 are saved as finds (target 14), with ≤ 6 false alarms per 100 m (target 3).
- Reliable upload: finds upload automatically when internet is available; cutting power mid-upload loses or duplicates none.
- Website: approved finds appear as map pins within ~10 m of the true spot, and a visitor can mark one "cleaned" from a phone.

Primary target (by Mar 20, 2027), self-driving

- The robot drives the 40–60 m home route by itself while I hold the safety button: ≥ 5 of 10 runs with no takeover (target 7), ≤ 2 takeovers per 100 m (target 1).
- Self-driving and litter detection run together at ≥ 5 frames per second and find ≥ 12 of 20 staged items.

Stretch (only if on schedule)

- One extra: auto-pause when a person is close, an alert on the robot for each find, or a weekly stats panel on the site.

4. Scope and Boundaries

Clarify what is explicitly included in this project and what is explicitly excluded, to prevent scope creep later.

> ***In scope:*** *the features, platforms, or use cases you commit to delivering by the deadline.*
>
> ***Out of scope:*** *related ideas you are consciously setting aside for a future version or a different project.*

In scope

- One robot on one fixed home route (40–60 m, ≥ 1.5 m wide, no stairs, road crossings, or forks), plus a separate stretch used only for testing.
- One detection class, "litter": items about 8 cm or larger (bottles, cans, cups, wrappers, bags, paper).
- Dry, daylight operation, with me walking behind holding the safety button.
- Store-and-forward upload: save on the robot, upload later over Wi-Fi or a phone hotspot.
- Website: map, photo page, "mark cleaned" button, password-protected approval page.
- Staged (clean, placed) litter for training and testing.

Out of scope

- Obstacle avoidance and driving near people (I stop the robot instead).
- Multiple routes, full-campus navigation, road crossings; rain or night driving.
- Picking litter up; tiny items such as cigarette butts and bottle caps.
- Facilities work-order integration, user accounts, notifications.

5. Technical Approach and Methodology

Outline your current best guess at the technologies, architecture, algorithms, or methods you expect to use, and how you will verify the result works.

> ***Consider:*** *languages/frameworks under consideration, major components or modules, data sources, and how you will test or evaluate correctness and performance.*
>
> ***It's okay if:*** *this section is tentative — flag the specific technical decisions still open and what you'd need to learn or prototype to resolve them.*

Hardware (~$360–455). Waveshare RaspRover robot base [4]: metal, four-wheel drive, motors with encoders (sensors that count wheel turns, so speed stays steady at a crawl). A Raspberry Pi 5 (credit-card-sized Linux computer) is the onboard computer, with the kit's 160° USB camera, a USB GPS receiver, and a wireless

gamepad. The base's motor board ships with its own firmware, so my Python code only sends short text commands over a serial (two-wire) link [5]. No embedded programming.

Software (one GitHub repo, four parts)

- **Robot code (Python, on the Pi):** driving; a safety state machine (named modes IDLE / GAMEPAD / AUTO / STOPPED with explicit switching rules); camera and GPS readers; a recorder; the two models; de-duplication (one bottle = one find); and an upload queue.
- **Vision models:** Ultralytics YOLO26n [2], a small, fast object detector, fine-tuned on ~350 of my own labeled campus photos plus the public TACO litter dataset [1]; and YOLO26n-seg, which outlines the sidewalk pixel by pixel, trained on ~200 outlined frames. Labeling in Roboflow [9], training on my laptop's RTX 4070 GPU, then conversion to the NCNN format, the fastest option on the Pi [3].
- **Self-driving:** the sidewalk model colors in the pavement; the code measures how far the pavement's center is from the image's center in a strip just ahead, and steers in proportion (a "P-controller"). Chosen over GPS navigation (3–10 m error vs. a 1.5 m sidewalk) and over networks that copy human driving (hard to debug).
- **Website:** Flask (a small Python web framework) [10] + SQLite (a one-file database) + Leaflet map [11], hosted on PythonAnywhere's free plan [8].
- **Connectivity:** outdoor campus Wi-Fi is poor, so the robot never needs internet while driving. It stores finds and uploads them once back on Wi-Fi or a phone hotspot.

Safety. The robot moves only while I hold a gamepad button (release = stop). It also stops if the controller signal or the sidewalk is lost for 0.5 s. Top speed when self-driving is 0.3 m/s. Testing happens only in quiet hours with written campus permission.

Evaluation

- Models: mAP50, precision, and recall on photos from a separate test stretch never used in training. Neighboring video frames are near-copies, so a random split would inflate scores.
- Field: 20 staged items per test; items found vs. false alarms; GPS pin error in meters.
- Driving: a 10-run trial with code frozen; completion rate and takeovers per 100 m.
- Code: pytest unit tests for the safety, steering, de-duplication, and upload logic. New steering code runs on recorded drives ("replay testing") before any outdoor test.

Open decisions (decide by · fallback)

- Is the kit camera sharp enough at walking pace? (Oct 24 · Pi Camera Module 3 Wide, ~$39)
- Does the public TACO data help, or only my own photos? (Jan 2027 · train both ways and compare)
- Is the Pi fast enough to run both models at ≥ 5 frames/s? (Jan 30 · smaller input size, or detect litter after the drive)
- Is self-driving reliable enough to demo? (Mar 7–13 · present the best recorded run plus a failure analysis)

Still to learn: PyTorch/Ultralytics training, Linux and SSH on the Pi, Flask, and basic steering (control) tuning.

6. Related Work and Existing Solutions

Briefly identify at least two or three existing tools, products, papers, or approaches that address a similar problem, and state how your project will differ or improve on them.

> ***Consider:*** *a short table or bullet list comparing each alternative's strengths and limitations against your intended approach.*

Existing approach	Strengths	Limitations	How LitterBot differs
Litterati app [6]	Large global community; geotagged photos build a litter database used by cities and companies	People must find, photograph, and tag every item by hand	The robot finds litter on its route automatically; people only do the cleanup step
Maintenance / 311-style reporting	Reports go straight to the staff responsible	Someone has to notice and bother to report; often no photo or exact location	Automatic photo + GPS; an approval queue filters out mistakes
TACO dataset and baseline [1]	Open, labeled litter photos (1,500 images, 60 categories)	Phone photos from standing height; authors say more data is needed; no working system	Adds my own ankle-height campus photos; runs on a $110 computer; measured in the field
Donkeycar [7]	Popular open-source Python platform for small self-driving cars	Learns by copying human driving (a black box); usually indoor tracks	Sidewalk outline + simple steering rule, so every step is visible and debuggable; outdoors

7. Timeline and Milestones

Break the semester into 4–6 milestones with target dates and a one-line deliverable for each, working backward from the final deadline.

Target Date	Milestone / Deliverable	Status
Oct 31, 2026	Robot built; gamepad driving with safety stop; campus video + GPS recorded; first baseline litter model scored	In progress
Dec 5, 2026	350 litter photos and 150 sidewalk outlines labeled (light month: research poster)	Not started
Jan 16, 2027	Website live: upload endpoint, approval page, public map, “mark cleaned” (built over winter break)	Not started
Feb 13, 2027	Minimum bar: both models on the robot; staged litter found while gamepad-driven appears on the public map	Not started
Mar 20, 2027	Self-driving on the home route; 10-run reliability trial; combined self-driving + detection run filmed	Not started

Target Date	Milestone / Deliverable	Status
Apr 16, 2027	Final presentation: live demo plus backup video; code tagged v1.0	Not started

Breaks: fall break and Dec 20–Jan 2 off; spring break is rest. Week-by-week plan: *docs/PLAN.md* in the repo.

8. Anticipated Challenges and Risks

Name the two or three biggest things that could go wrong (technical, scheduling, data access, scope) and your current plan for handling each.

> ***Consider:*** *a simple risk / mitigation pairing rather than a general worry — e.g., "Risk: needed dataset may be restricted. Mitigation: identify a public substitute by week 3."*

Risk	Mitigation
Weather: Oregon rain and early darkness (Oct–Apr) limit outdoor testing.	Record all training data in the dry Sep–Oct window; develop steering on recorded drives indoors; every outdoor week has an indoor backup task.
Self-driving isn't reliable by March.	The minimum bar (gamepad-driven pipeline) is complete Feb 13 without it; one simple route; on Mar 7–13, if below 5/10, present the best run plus a failure analysis.
Model accuracy: small dataset, and public photos look different from the robot's low view.	Train mostly on my own robot-height photos; include ~50 litter-free photos to cut false alarms; study the 20 worst mistakes. The approval queue keeps false alarms off the public map. If mAP50 < 0.50, narrow to bottles, cans, and cups.
Schedule: research poster in November, grad applications, job.	~4.75 h/week in normal weeks, capped at 5.5; light Nov–Jan; a pre-ranked list of scope cuts instead of heavy weeks.

9. Expected Outcomes and Deliverables

List the concrete artifacts you will hand in at the end of the semester, such as source code, a deployed demo, a final report, a poster, or a dataset.

- Working robot and all source code (GitHub: EthanBucy/Capstone, AGPL-3.0 license).
- Two trained models with a model card (training data, scores, known failures).
- Deployed public website with map, plus campus QR posters.
- Test logs: model scores, field tests, and the 10-run self-driving trial.
- Labeled campus litter and sidewalk datasets (public on Roboflow).
- Documentation: project plan, setup guides, decision notes.
- Final presentation (Apr 16, 2027) with live demo and a 90-second backup video.

10. Resources and Support Needed

Note any hardware, software licenses, datasets, cloud credits, faculty expertise, or external partners this project will require.

- **Hardware (~$360–455, self-funded, under $500):** RaspRover base $174.99, Raspberry Pi 5 4 GB $110, 3 × 18650 batteries, USB GPS, gamepad, memory card, battery-safe charging bag. **Open question:** can the department reimburse any of this?
- **Computing:** my Windows laptop with an NVIDIA RTX 4070 (training) and my Mac (development). No cloud credits needed.
- **Software and services (all free):** Ultralytics YOLO, PyTorch, OpenCV, Flask, Roboflow free plan, PythonAnywhere free plan, GitHub, Notion.
- **Data:** TACO dataset (CC BY 4.0) and my own campus photos.
- **Permissions:** Campus Safety/Facilities approval to test on sidewalks; approval to hang posters.
- **Faculty:** monthly advisor check-ins; ideally a faculty member with ML or robotics experience for occasional design sanity checks.

11. References

Cite any articles, tools, APIs, or prior work referenced above, in a consistent citation style (e.g., IEEE or ACM).

[1] P. F. Proença and P. Simões, “TACO: Trash Annotations in Context for Litter Detection,” arXiv:2003.06975, 2020.

[2] Ultralytics, “Ultralytics YOLO Docs,” 2026. [Online]. Available: https://docs.ultralytics.com

[3] Ultralytics, “Quick Start Guide: Raspberry Pi with Ultralytics YOLO26,” 2026. [Online]. Available: https://docs.ultralytics.com/guides/raspberry-pi/

[4] Waveshare, “RaspRover Open-source 4WD AI Robot,” 2026. [Online]. Available: https://www.waveshare.com/rasprover.htm

[5] Waveshare, “WAVE ROVER Wiki,” 2026. [Online]. Available: https://www.waveshare.com/wiki/WAVE_ROVER

[6] Litterati, “Litterati.” [Online]. Available: https://www.litterati.org

[7] Donkeycar, “Donkeycar: a Python self driving library,” GitHub. [Online]. Available: https://github.com/autorope/donkeycar

[8] PythonAnywhere, “Free Accounts Features,” 2026. [Online]. Available: https://help.pythonanywhere.com/pages/FreeAccountsFeatures/

[9] Roboflow, “Roboflow Docs,” 2026. [Online]. Available: https://docs.roboflow.com

[10] Pallets Projects, “Flask Documentation,” 2026. [Online]. Available: https://flask.palletsprojects.com

[11] V. Agafonkin, “Leaflet: an open-source JavaScript library for interactive maps.” [Online]. Available: https://leafletjs.com